MATH 189 - Homework 3

due Wednesday, February 4 at 8:00AM

Resources: These problems appear in Chs. 4-5 of the textbook pgs. 193-199 and 224-228.

Format requirements: Type your homeworks using LaTeXwith embedded plots for the applied exercises.

Grading: The grading is based 50% on correctness and 50% on correctness.

Problem 1. Ch. 4 conceptual exercise 7

Suppose that we wish to predict whether a given stock will issue a dividend this year ("Yes" or "No") based on X, last year's percent profit. We examine a large number of companies and discover that the mean value of X for companies that issued a dividend was $\bar{X} = 10$, while the mean for those that didn't was $\bar{X} = 0$. In addition, the variance of X for these two sets of companies was $\hat{\sigma}^2 = 36$. Finally, 80% of companies issued dividends. Assuming that X follows a normal distribution, predict the probability that a company will issue a dividend this year given that its percentage profit was $X = 4$ last year.

Solution: Using Bayes' theorem, we can find the probability that a company will issue a dividend this year given that its percentage profit was $X = 4$ last year. We know that $P(Y = \text{Yes}) = 0.8$ and $P(Y = \text{No}) = 0.2$. We also know that $P(X = 4|Y = \text{Yes}) = \frac{1}{72\pi}e^{-\frac{(4-10)^2}{72}} \approx 0.00268$ and $P(X = 4|Y = \text{No}) = \frac{1}{72\pi}e^{-\frac{(4-0)^2}{72}} \approx 0.00354$. Thus,

$$P(Y = \text{Yes}|X = 4) = \frac{P(X = 4|Y = \text{Yes})P(Y = \text{Yes})}{P(X = 4)} = \frac{0.00268 \cdot 0.8}{0.00268 \cdot 0.8 + 0.00354 \cdot 0.2} \approx 0.752$$

So the probability that a company will issue a dividend this year given that its percentage profit was $X = 4$ last year is approximately 0.752.

Problem 2. Ch. 4 conceptual exercise 8

Suppose that we take a data set, divide it into equally-sized training and test sets, and then try out two different classification procedures. First we use logistic regression and get an error rate of 20% on the training data and 30% on the test data. Next we use 1-nearest neighbors (i.e. $K = 1$) and get an average error rate (averaged over both test and training data sets) of 18%. Based on these results, which method should we prefer to use for classification of new observations? Why?

Solution: We should prefer logistic regression for classification of new observations. The test error rate of 30% is more promising than the average error rate of 18% for 1-nearest neighbors ($K = 1$) because 1-NNs will generally overfit since they only care about one neighbor, usually achieving close to 0% training error. This indicates that the test error rate is closer to 36% which is worse than the 30% for logistic regression.

Problem 3. Ch. 5 conceptual exercise 3

We now review k-fold cross-validation.

a Explain how k-fold cross-validation is implemented.

> **Solution:** k-fold cross-validation is implemented by dividing the data into k equally sized folds. Then, for each fold, we train the model on the remaining $k-1$ folds and test on the fold. We then average the error rates over all k folds.

b What are the advantages and disadvantages of k-fold cross-validation relative to:

i. The validation set approach?

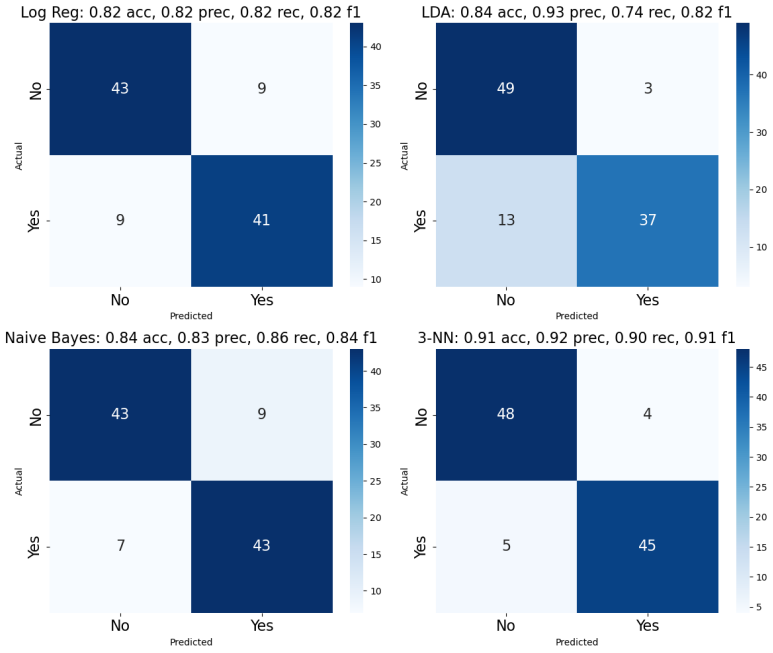
ii. Leave-one-out cross-validation?

> **Solution:** k-fold cross-validation is more efficient than leave-one-out cross-validation because you're not training on each data point $n-1$ times for n data points. A disadvantage is that it may not be as accurate as leave-one-out cross-validation because certain folds may be more representative of the data than others and will hence bias the results.
>
> k-fold cross-validation is also more accurate than the validation set approach because it ultimately tests on all of the data points, not just a subset. A disadvantage is that it is more computationally expensive than the validation set approach because you're training on $k-1$ times per data point for k folds.

Problem 4. Ch. 4 applied exercise 16

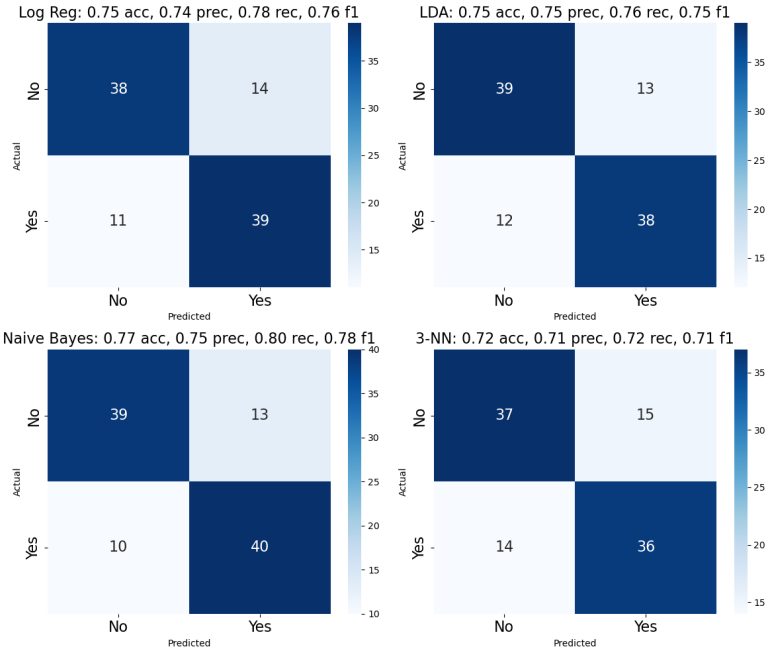
Using the `Boston` data set, fit classification models in order to predict whether a given suburb has a crime rate above or below the median. Explore logistic regression, LDA, naive Bayes, and KNN models using various subsets of the predictors. Describe your findings. Hint: You will have to create the response variable yourself, using the variables that are contained in the `Boston` data set.

Solution:



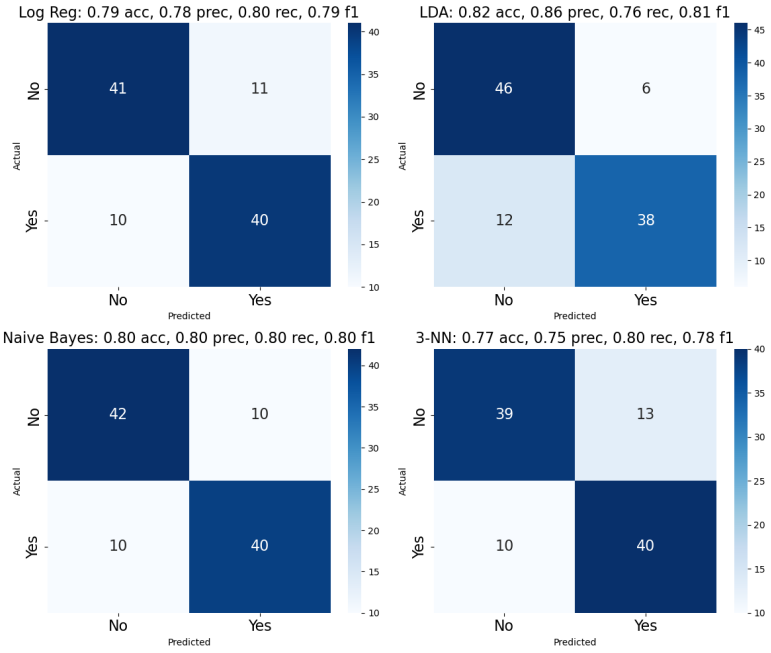
This confusion matrix is from all the predictors. As you can see from the confusion matrices and the metrics, Log Reg and Naive Bayes perform about equally well with all metrics being around 0.82-0.84 with NB being slightly better.

LDA performs better in terms of precision (0.93) but worse in recall (0.74). The F1 score is about the same being 0.82 so LDA is the choice if you need low false positives. The 3-NN performs the best overall with all metrics being 0.90 or higher.



This confusion matrix is from the socioeconomic predictors, i.e. `lstat`, `medv`, and `ptratio`. In this case, the 3-NN performs the worst overall indicating that whatever gave it the edge before wasn't a socioeconomic predictor.

LDA and Log Reg perform about equally well with all metrics being around 0.74-0.78 with Log Reg having an edge with recall being 0.78. Overall, Naive Bayes performs the best overall with metrics running from 0.75-0.80 with particular emphasis on recall being 0.80.



This confusion matrix is from the location predictors, i.e. `dis`, `nox`, and `chas`. In this case, Naive Bayes and Log Reg perform about equally. LDA is back to having high precision (0.86) but low recall (0.76). The 3-NN also performed relatively poorly. Overall, these predictors surprisingly performed a bit better than the socioeconomic predictors indicating that location is more important in predicting crime rate.

Problem 5. Ch. 5 applied exercise 9

We will again consider the `Boston` housing data set, from the ISLP library.

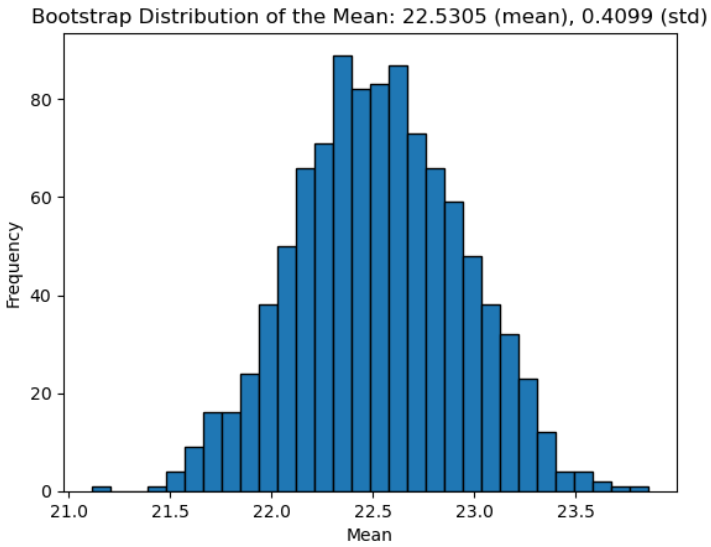
a Based on this data set, provide an estimate for the population mean of `medv`. Call this estimate $\hat{\mu}$.

> **Solution:** We predict $\hat{\mu} = 22.5328$ for the population mean of `medv`.

b Provide an estimate of the standard error of $\hat{\mu}$. Interpret this result. Hint: We can compute the standard error of the sample mean by dividing the sample standard deviation by the square root of the number of observations.

> **Solution:** We estimate the standard error of $\hat{\mu}$ to be 0.4089. This means that the sample mean is likely to be within 0.4089 of the true population mean.

c Now estimate the standard error of $\hat{\mu}$ using the bootstrap. How does this compare to your answer from (b)?

> **Solution:**
>
> 
>
>
> The bootstrap estimate of the standard error of $\hat{\mu}$ is 0.4099 which is very close to the answer from (b) since it's only off by 0.001.

d Based on your bootstrap estimate from (c), provide a 95% confidence interval for the mean of `medv`. Compare it to the results obtained by using `Boston['medv'].std()` and the two standard error rule. Hint: You can approximate a 95% confidence interval using the formula $[\hat{\mu} - 2SE(\hat{\mu}), \hat{\mu} + 2SE(\hat{\mu})]$.

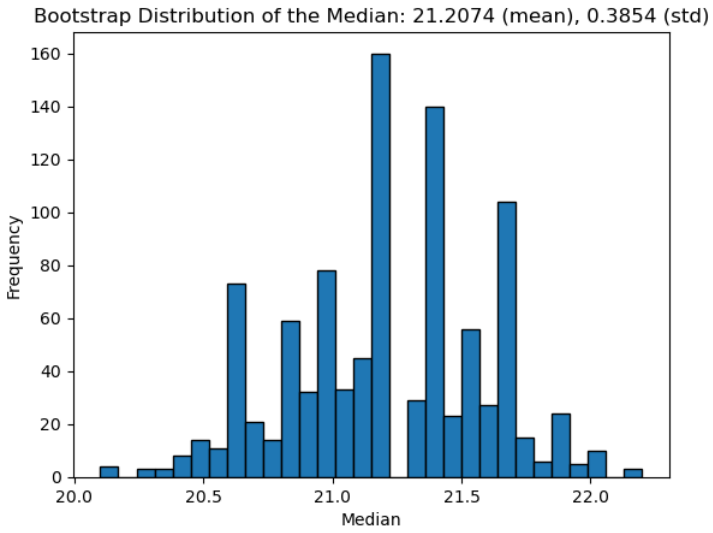
> **Solution:** Using the bootstrap estimate, the 95% confidence interval is $22.5305 \pm 2(0.4099) = [21.7107, 23.3503]$. Using our results from (b), the 95% confidence interval is $22.5328 \pm 2(0.4089) = [21.7150, 23.3506]$. These intervals are extremely similar and only differ by less than 0.01.

e Based on this data set, provide an estimate, $\hat{\mu}_{med}$, for the median value of `medv` in the population.

Solution: We predict $\hat{\mu}_{med} = 21.2$ for the median value of `medv` in the population.

f We now would like to estimate the standard error of $\hat{\mu}_{med}$. Unfortunately, there is no simple formula for computing the standard error of the median. Instead, estimate the standard error of the median using the bootstrap. Comment on your findings.

Solution:



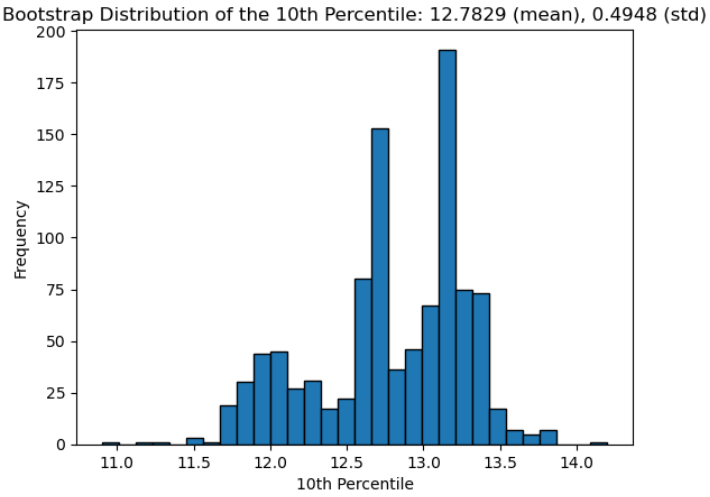
The bootstrap estimate of the standard error of $\hat{\mu}_{med}$ is 0.3854. This is smaller than the standard error of $\hat{\mu}$ indicating that the median is more stable than the mean. This makes sense since the median is not as affected by outliers as the mean.

g Based on this data set, provide an estimate for the tenth percentile of `medv` in Boston census tracts. Call this quantity $\hat{\mu}_{0.1}$. (You can use the `np.percentile()` function.)

Solution: We predict $\hat{\mu}_{0.1} = 12.75$ for the tenth percentile of `medv` in the population.

h Use the bootstrap to estimate the standard error of $\hat{\mu}_{0.1}$. Comment on your findings

Solution:



Interestingly, the bootstrap estimate of the standard error of $\hat{\mu}_{0.1}$ is 0.4948 which is even larger than the standard error of the mean. This does make somewhat sense though since there will be more variablility in the tails of the distribution than the center.

Problem 4

In [104…
```
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from ISLP import load_data

Boston = load_data('Boston')
```

In [97]:
```
Boston["Y"] = Boston["crim"] > Boston["crim"].median()
X_all = Boston.drop(columns=["Y", "crim"])
X_socio = Boston[["lstat", "medv", "ptratio"]]
X_loc = Boston[["dis", "nox", "chas"]]
```

In [98]:
```
def plot_confusion_matrix(Y_test, Y_pred, model_name, ax=None):
    cm = confusion_matrix(Y_test, Y_pred)
    acc = accuracy_score(Y_test, Y_pred)
    prec = precision_score(Y_test, Y_pred)
    rec = recall_score(Y_test, Y_pred)
    f1 = f1_score(Y_test, Y_pred)

    if ax is None:
        _, ax = plt.subplots(figsize=(8, 6))

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['No', 'Yes'], y
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
    ax.tick_params(axis='both', which='major', labelsize=16)
    ax.set_title(f'{model_name}: {acc:.2f} acc, {prec:.2f} prec, {rec:.2f} rec, {f1

    if ax is None:
        plt.show()
    return ax
```

In [99]:
```
def train_models_and_plot(X):
    Y = Boston["Y"]
    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random

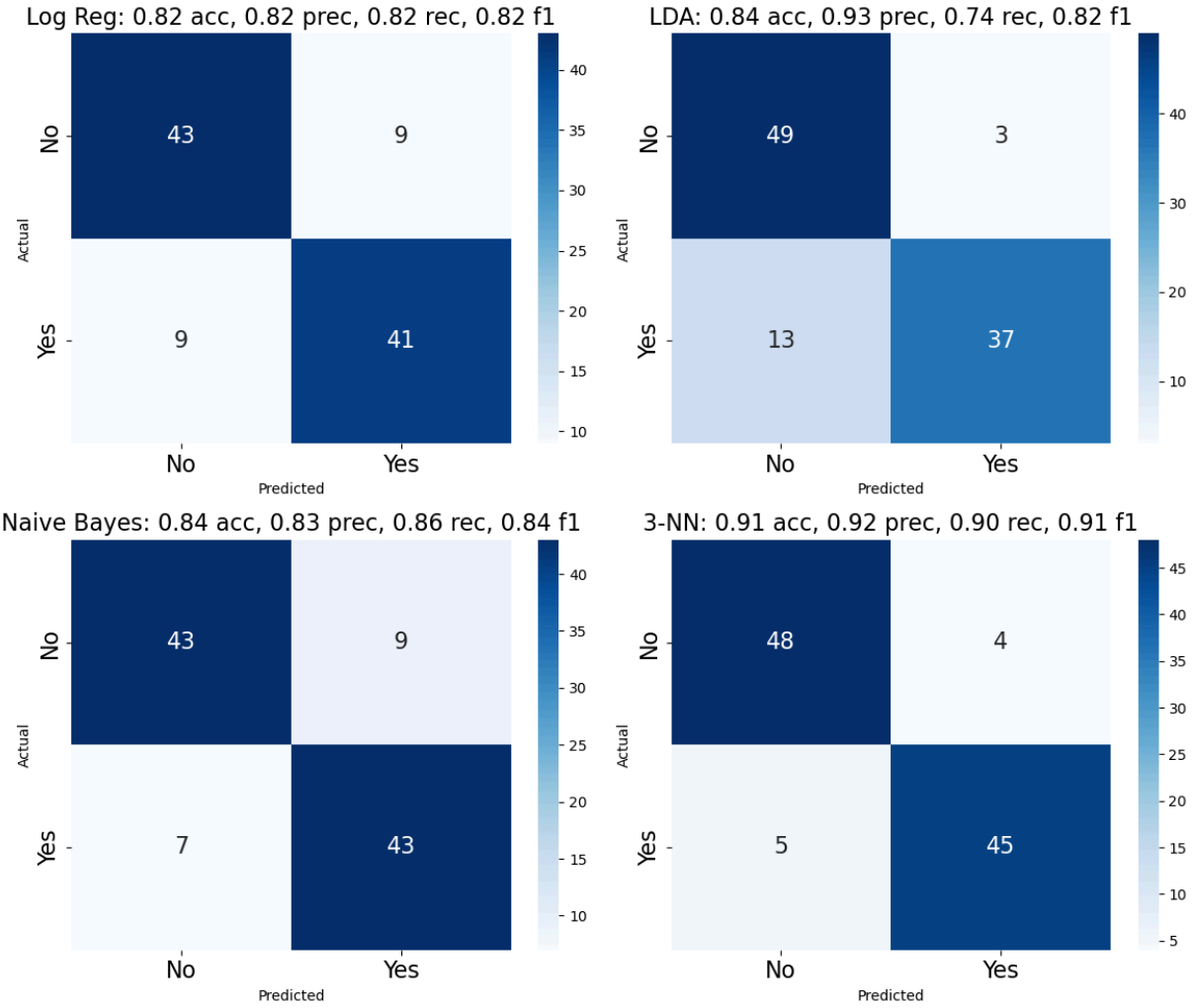
    models = {
        "Log Reg": LogisticRegression(max_iter=100_000),
        "LDA": LinearDiscriminantAnalysis(),
        "Naive Bayes": GaussianNB(),
```

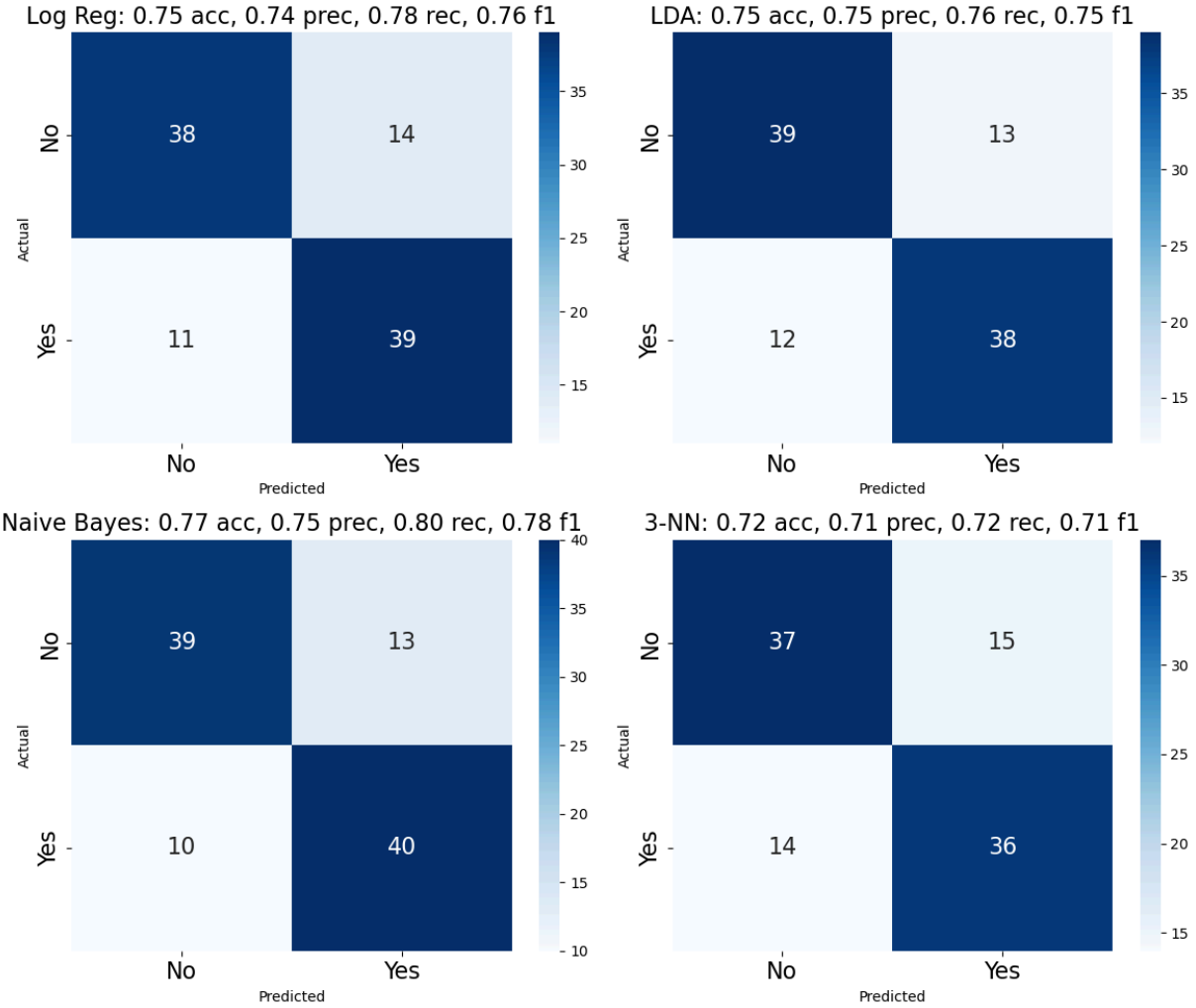
```
        "3-NN": KNeighborsClassifier(n_neighbors=3)
    }

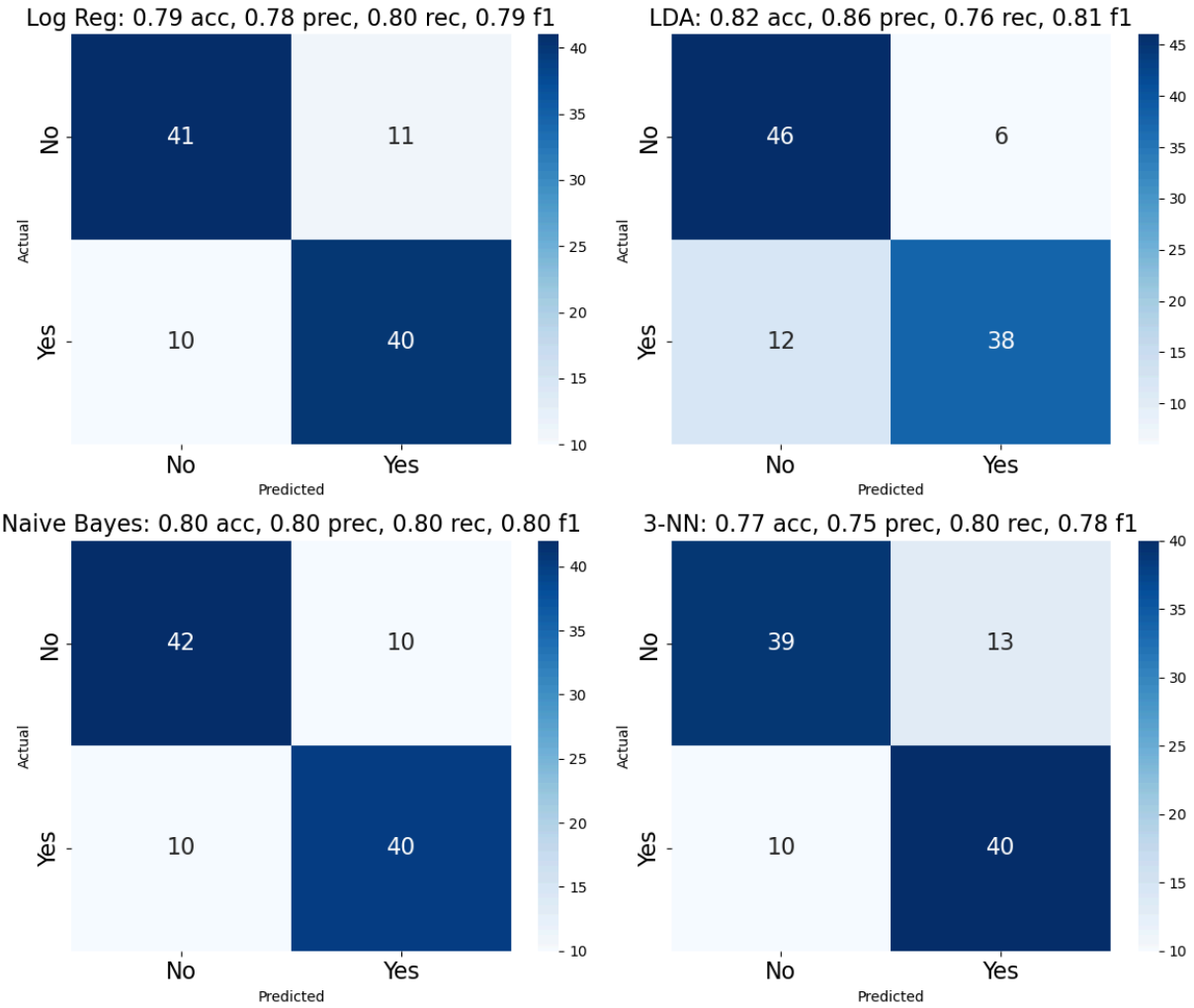
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    for ax, (model_name, model) in zip(axes.flatten(), models.items()):
        model.fit(X_train, Y_train)
        Y_pred = model.predict(X_test)
        plot_confusion_matrix(Y_test, Y_pred, model_name, ax=ax)
    plt.tight_layout()
    plt.show()
```

In [100…

```
train_models_and_plot(X_all)
train_models_and_plot(X_socio)
train_models_and_plot(X_loc)
```




Log Reg: 0.75 acc, 0.74 prec, 0.78 rec, 0.76 f1
38
14
11
39
No
Yes
Actual
Predicted
LDA: 0.75 acc, 0.75 prec, 0.76 rec, 0.75 f1
39
13
12
38
Naive Bayes: 0.77 acc, 0.75 prec, 0.80 rec, 0.78 f1
39
13
10
40
3-NN: 0.72 acc, 0.71 prec, 0.72 rec, 0.71 f1
37
15
14
36



Problem 5

In [ ]:
```
# 5a
Boston["medv"].mean()
```

Out[ ]:
```
np.float64(22.532806324110677)
```

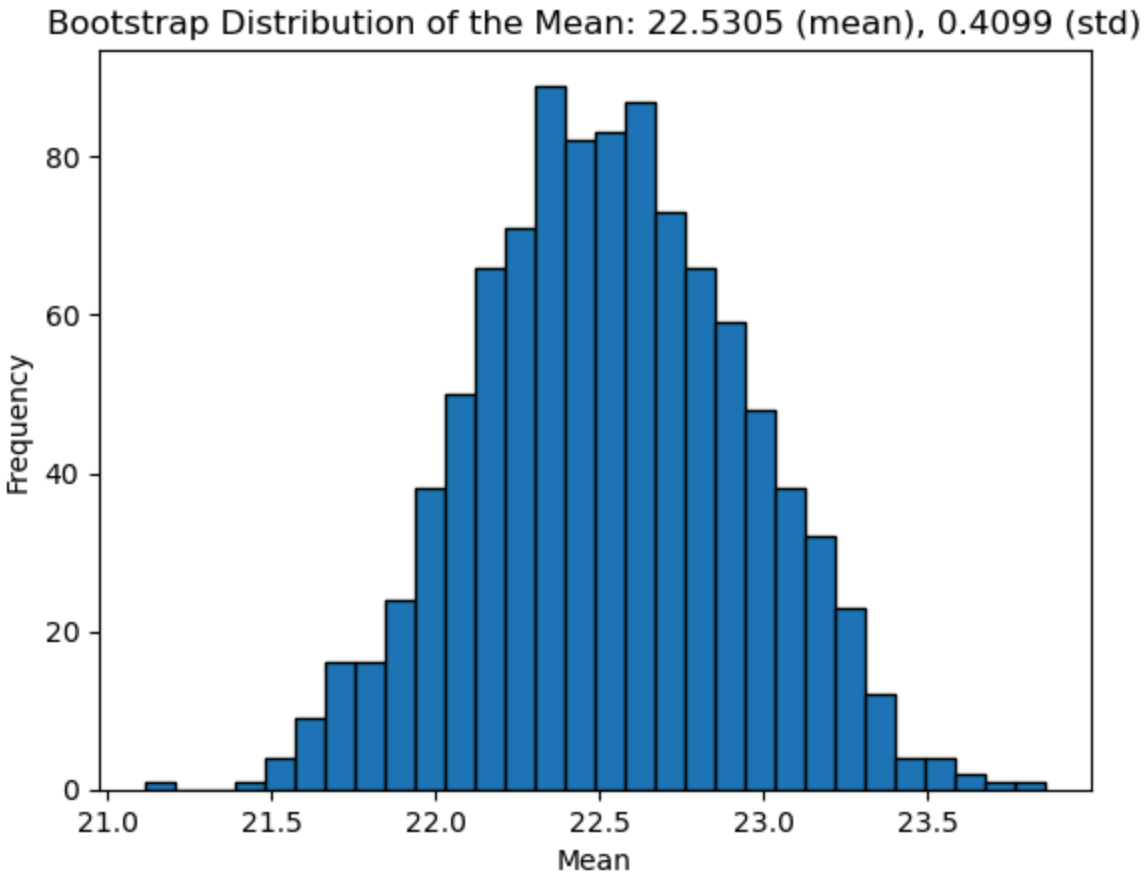
In [ ]:
```
# 5b
Boston["medv"].std() / np.sqrt(Boston.shape[0])
```

Out[ ]:
```
np.float64(0.40886114749753505)
```

In [109…
```
# 5c

bootstrap_means = []
for i in range(1000):
    bootstrap_sample = Boston["medv"].sample(n=Boston.shape[0], replace=True)
    bootstrap_mean = bootstrap_sample.mean()
    bootstrap_means.append(bootstrap_mean)
bootstrap_means = np.array(bootstrap_means)
plt.hist(bootstrap_means, bins=30, edgecolor='black')
plt.title(f"Bootstrap Distribution of the Mean: {bootstrap_means.mean():.4f} (mean)
```

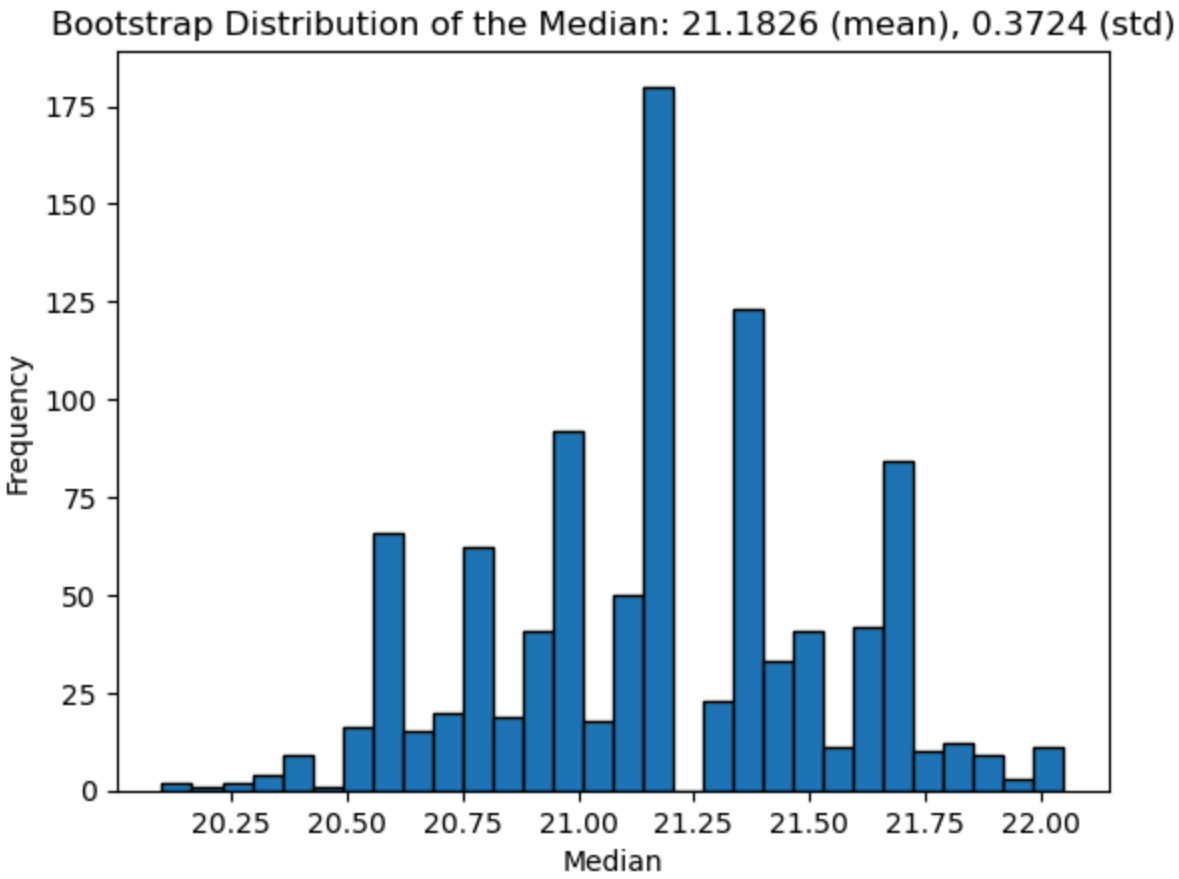
```
plt.xlabel(f"Mean")
plt.ylabel(f"Frequency")
plt.show()
```



In [ ]:
```
# 5e
Boston["medv"].median()
```

Out[ ]:
```
np.float64(21.2)
```

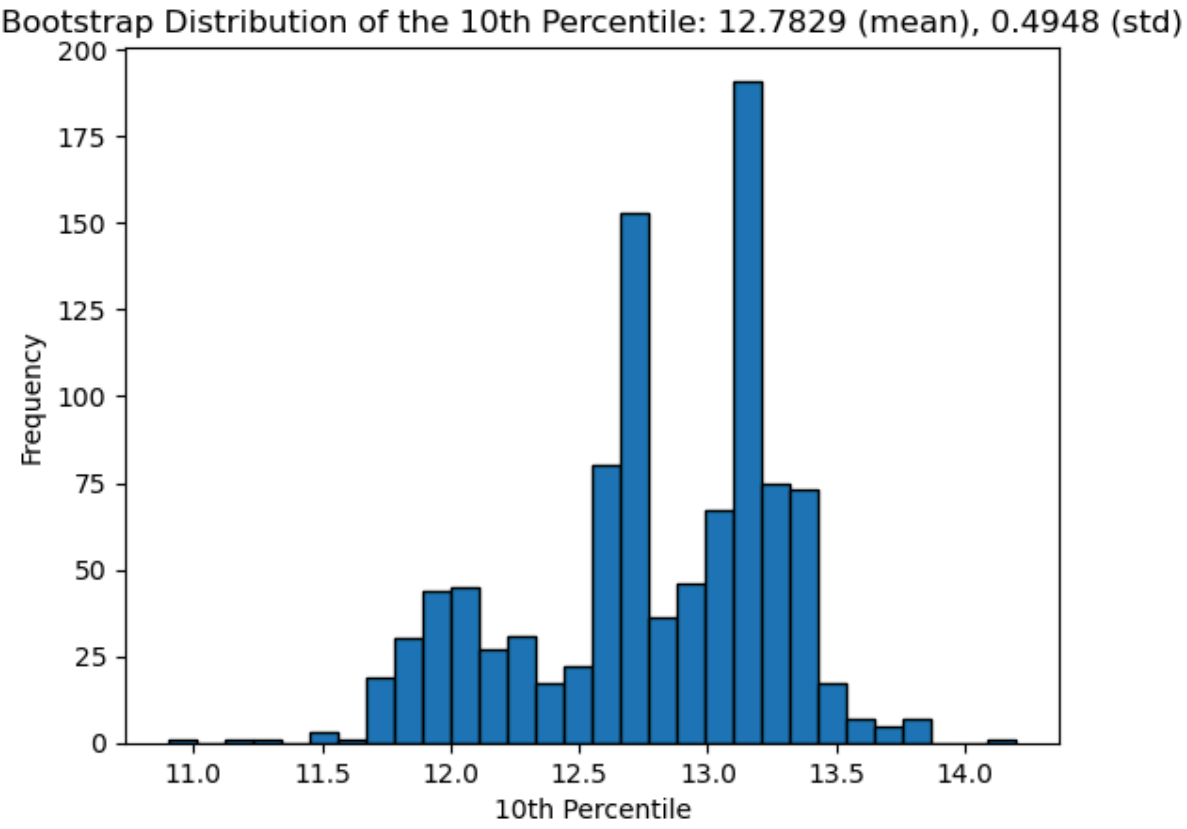
In [112…
```
# 5f
bootstrap_medians = []
for i in range(1000):
    bootstrap_sample = Boston["medv"].sample(n=Boston.shape[0], replace=True)
    bootstrap_median = bootstrap_sample.median()
    bootstrap_medians.append(bootstrap_median)
bootstrap_medians = np.array(bootstrap_medians)
plt.hist(bootstrap_medians, bins=30, edgecolor='black')
plt.title(f"Bootstrap Distribution of the Median: {bootstrap_medians.mean():.4f} (m
plt.xlabel(f"Median")
plt.ylabel(f"Frequency")
plt.show()
```



In [113…
```
# 5g
np.percentile(Boston["medv"], 10)
```

Out[113… np.float64(12.75)

In [114…
```
# 5h
bootstrap_percentiles = []
for i in range(1000):
    bootstrap_sample = Boston["medv"].sample(n=Boston.shape[0], replace=True)
    bootstrap_percentile = np.percentile(bootstrap_sample, 10)
    bootstrap_percentiles.append(bootstrap_percentile)
bootstrap_percentiles = np.array(bootstrap_percentiles)
plt.hist(bootstrap_percentiles, bins=30, edgecolor='black')
plt.title(f"Bootstrap Distribution of the 10th Percentile: {bootstrap_percentiles.m
plt.xlabel(f"10th Percentile")
plt.ylabel(f"Frequency")
plt.show()
```


Bootstrap Distribution of the 10th Percentile: 12.7829 (mean), 0.4948 (std)
Frequency
10th Percentile
0
25
50
75
100
125
150
175
200
11.0
11.5
12.0
12.5
13.0
13.5
14.0